BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2
NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN, ĐIỆN TỬ

BÁO CÁO TIẾN ĐỘ 2

MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI:

TÌM VỊ TRÍ ĐẦU, CUỐI VÀ TẦN SUẤT CỦA X

Mã đề: DT028

Giảng viên hướng dẫn	:	Hồ Nhựt Minh
Sinh viên thực hiện	:	PHAN ĐỨC HUY
MSSV	:	N24DCDT034
Lớp	:	D24CQDT01-N

TP. HCM, tháng 10 năm 2026

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Môi trường: Windows 11; ngôn ngữ Julia phiên bản 1.13.1 (chỉ thư viện chuẩn); trình soạn thảo Visual Studio Code 1.142.0 với terminal PowerShell. Mã chính tại src/DeTai.jl, demo tại demo.jl, bộ test tại test/runtests.jl. SHA chạy C và SHA nộp S do sinh viên ghi sau khi đẩy GitHub, theo hướng dẫn ở mục 4.9 và tệp ThongTinNop.txt.

4.1.2. Phương pháp xác định kỳ vọng

Kỳ vọng được tính độc lập bằng tay (giải từng bước) và đối chiếu bằng quét tuyến tính độc lập trong kiểm thử. Với kết quả nguyên, so sánh trực tiếp không cần sai số. Mỗi ca ghi rõ input đủ tái hiện, expected, actual và kết luận PASS/FAIL.

4.2. Bảng kết quả 13 ca

Bảng dưới liệt kê 13 ca có mã: năm ca trên phiếu (gồm hai ví dụ và ba ca biên), hai ca tự thiết kế, một ca input sai và năm ca bổ sung. Bộ kiểm thử còn sinh thêm mảng ngẫu nhiên có seed cố định để kiểm bất biến.

Miền chính là mảng số nguyên tăng; các ca biên và ca hàm con (validator, quét tuyến tính, đếm trong đoạn) được dùng để kiểm nhánh đặc biệt của thuật toán.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	[1,2,2,2,4], x = 2 → 2, 4, 3	VD01 / A01	01_vi_du_1.png
Ca phiếu 2	Cùng mảng, x = 3 → 0, 0, 0	VD02 / A02	02_vi_du_2.png
Ca phiếu 3	[5,5,5], x = 5 → 1, 3, 3	BIEN03 / A07	07_bien_3.png
Ca phiếu 4	[1,2], x = 1 → 1, 1, 1	BIEN01 / A05	05_bien_1.png
Ca phiếu 5	Mảng rỗng → 0, 0, 0	BIEN02 / A06	06_bien_2.png

Mỗi ca phiếu được giữ nguyên input và kỳ vọng; các ca trùng mục tiêu được gộp mã khi minh chứng nhưng vẫn kiểm riêng trong bộ test. Ca 1 và ca 2 dùng chung mảng để thấy rõ khác biệt giữa có mặt và vắng mặt.

Bảng 4.2 Thiết kế 13 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ 1 phiếu / tan_suat	[1,2,2,2,4], x=2	first=2, last=4, count=3; xét mid=6
VD02	Ví dụ 2 phiếu / tan_suat	[1,2,2,2,4], x=3	0,0,0; xét mid=6; không nhận vị trí chèn
TK01	Tự thiết kế 1 / vi_tri_dau	[1,1,1,5,9], x=1	biên trái ở 1; count=3
TK02	Tự thiết kế 2 / vi_tri_cuoi	[0,3,7,7], x=7	biên phải ở 4; count=2
BIEN01	Ca phiếu 4 / tan_suat	[1,2], x=1	1,1,1; count=1
BIEN02	Ca phiếu 5 / tan_suat	Int[], x=5	0,0,0; 0 lần xét mid
BIEN03	Ca phiếu 3 / tan_suat	[5,5,5], x=5	1,3,3; toàn mảng bằng x
LOI01	Input sai / kiem_tra_dau_vao	[3,1,2], x=1	ném ArgumentError (mảng không tăng)
KT09	Một phần tử / tan_suat	[5], x=5	1,1,1; xét mid=2
KT10	Nhỏ hơn mọi phần tử / tan_suat	[2,4,6], x=1	0,0,0; lo vượt hi
KT11	Lớn hơn mọi phần tử / tan_suat	[2,4,6], x=9	0,0,0; lo vượt hi (lo = n+1)
KT12	Khóa trùng giữa / tan_suat	[1,3,3,3,3,5,7,7], x=3	first=2, last=5, count=4
KT13	Số âm / tan_suat	[-5,-5,-2,0], x=-5	first=1, last=2, count=2

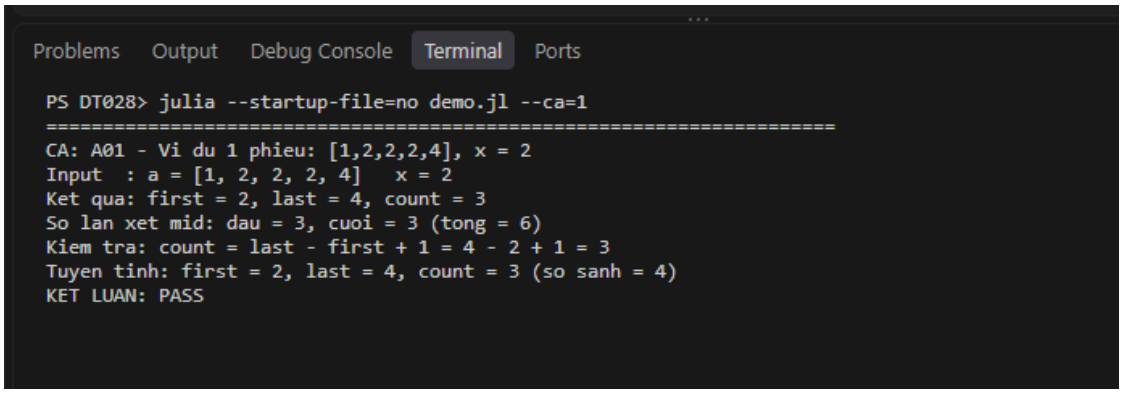
Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	(2, 4, 3)	PASS	A01 / 01_vi_du_1.png
VD02	(0, 0, 0)	PASS	A02 / 02_vi_du_2.png
TK01	(1, 3, 3)	PASS	A03 / 03_tu_thiet_ke_1.png
TK02	(3, 4, 2)	PASS	A04 / 04_tu_thiet_ke_2.png
BIEN01	(1, 1, 1)	PASS	A05 / 05_bien_1.png
BIEN02	(0, 0, 0)	PASS	A06 / 06_bien_2.png
BIEN03	(1, 3, 3)	PASS	A07 / 07_bien_3.png
LOI01	ArgumentError	PASS	A08 / 08_input_khong_hop_le.png
KT09	(1, 1, 1)	PASS	09_tong_ket_kiem_thu.png
KT10	(0, 0, 0)	PASS	09_tong_ket_kiem_thu.png
KT11	(0, 0, 0)	PASS	09_tong_ket_kiem_thu.png
KT12	(2, 5, 4)	PASS	09_tong_ket_kiem_thu.png
KT13	(1, 2, 2)	PASS	09_tong_ket_kiem_thu.png

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

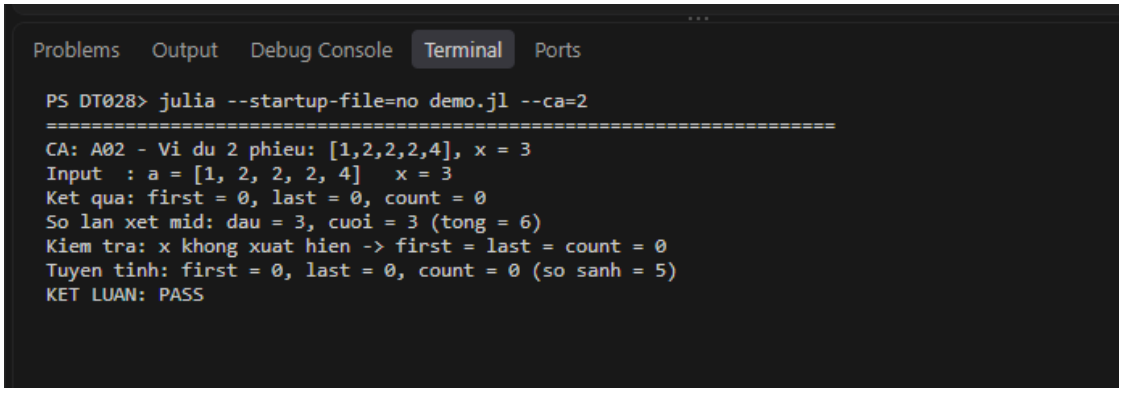
Input: [1,2,2,2,4], x = 2. Expected: first = 2, last = 4, count = 3, xác định bằng giải tay. Lệnh chạy: julia --startup-file=no demo.jl --ca=1. Actual: (2, 4, 3), 6 lần xét mid; quét tuyến tính cũng cho (2, 4, 3).



Hình 4.1 — Ví dụ 1 (VD01): output thật từ demo.jl --ca=1 trong VS Code.

4.3.2. Ví dụ 2 — VD02

Input: [1,2,2,2,4], x = 3. Expected: (0,0,0) vì x không có trong mảng. Lệnh chạy: julia --startup-file=no demo.jl --ca=2. Actual: (0,0,0), 6 lần xét mid. Ca này phân biệt vị trí chèn (sau phần tử 2) với một lần xuất hiện thật.

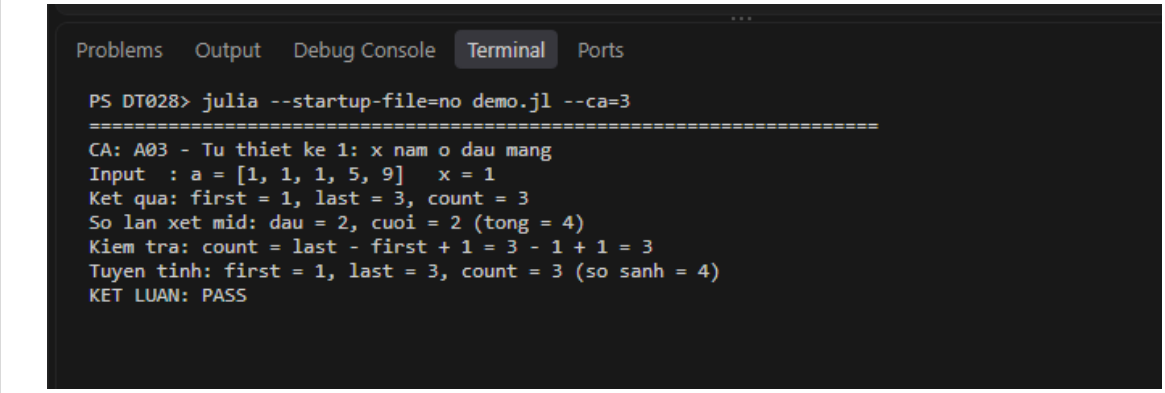


Hình 4.2 — Ví dụ 2 (VD02): x vắng mặt, không nhận vị trí chèn.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

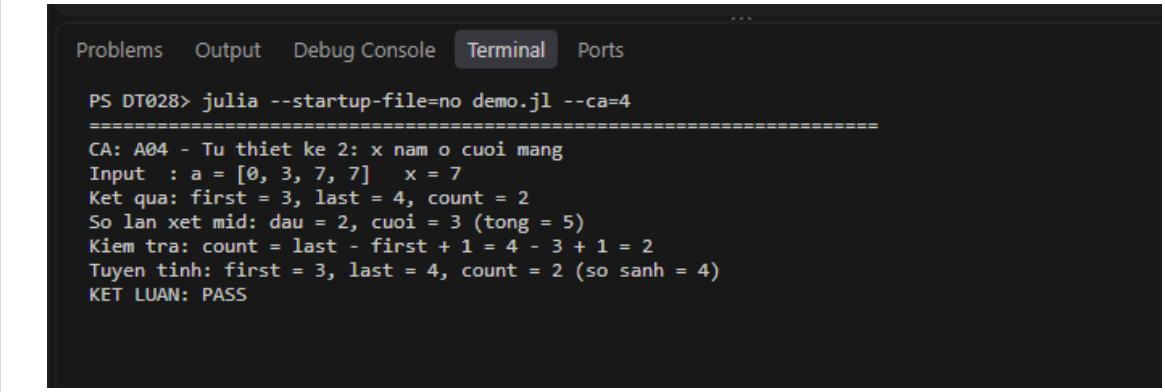
Mục đích: kiểm biên trái khi x nằm ở đầu mảng. Input [1,1,1,5,9], x = 1; expected first = 1, last = 3, count = 3. Actual khớp, 4 lần xét mid cho cả hai lần tìm.



Hình 4.3 — Ca tự thiết kế 1 (TK01): x ở đầu mảng.

4.4.2. Ca TK02

Mục đích: kiểm biên phải khi x nằm ở cuối mảng. Input [0,3,7,7], x = 7; expected first = 3, last = 4, count = 2. Actual khớp; bản tìm cuối phải xét thêm mid = 4.

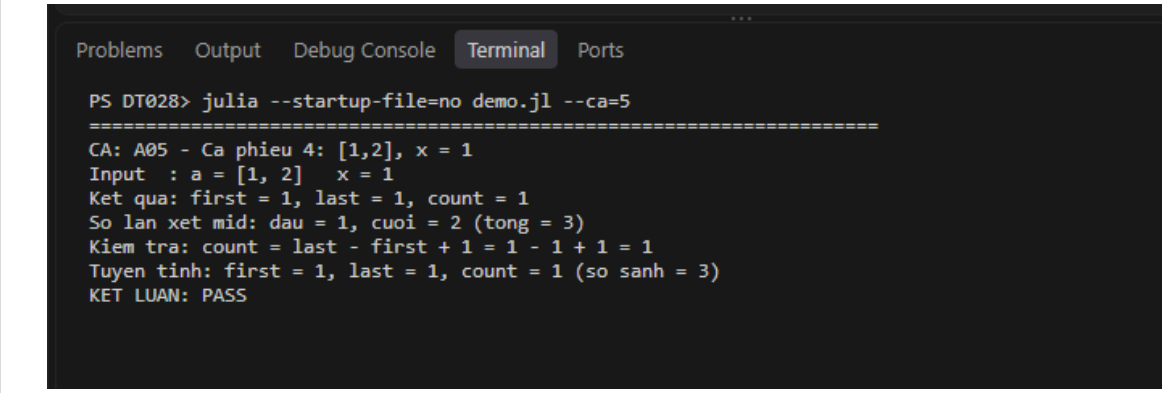


Hình 4.4 — Ca tự thiết kế 2 (TK02): x ở cuối mảng.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

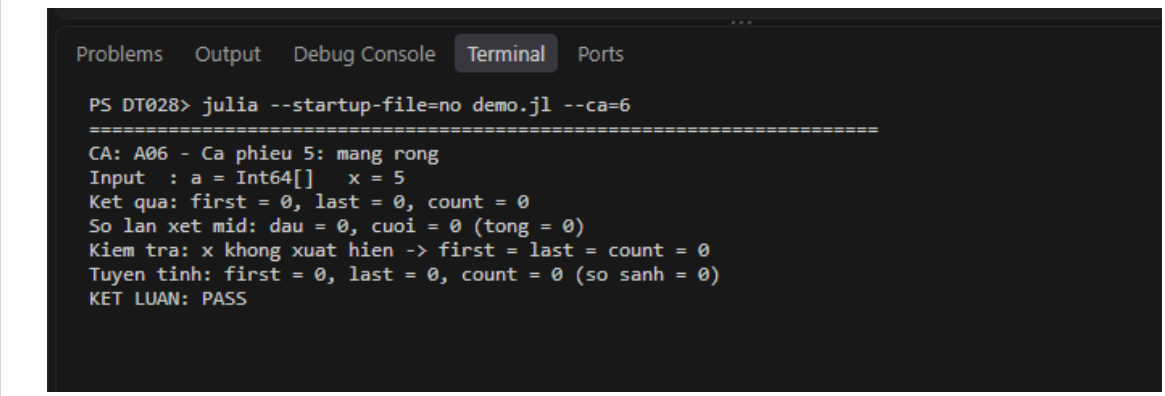
Biên theo phiếu: [1,2], x = 1. Expected (1,1,1): x ở ngay đầu mảng, không có khóa trùng. Actual khớp; bản tìm đầu chỉ 1 lần xét, bản tìm cuối 2 lần.



Hình 4.5 — Biên 1 (BIEN01): mảng hai phần tử.

4.5.2. Ca BIEN02

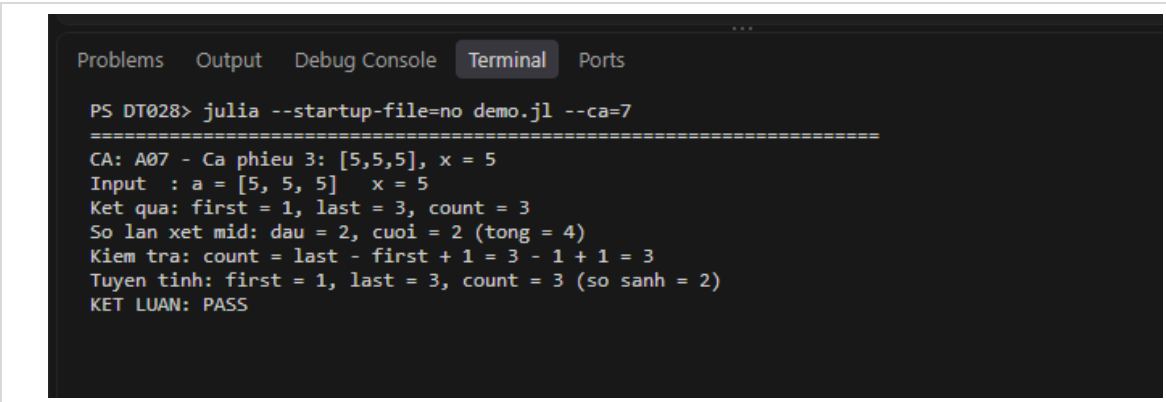
Biên: mảng rỗng Int[], x = 5 (ca phiếu 5). Expected (0,0,0) và 0 lần xét mid. Actual khớp; chương trình không truy cập a[mid] khi n = 0.



Hình 4.6 — Biên 2 (BIEN02): mảng rỗng.

4.5.3. Ca BIEN03

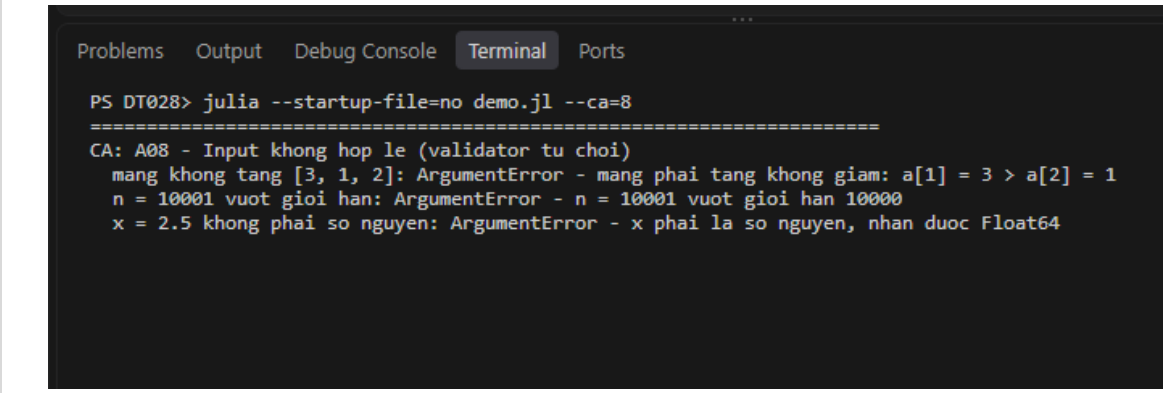
Biên: toàn bộ mảng bằng x, [5,5,5], x = 5 (ca phiếu 3). Expected (1,3,3): hai biên là hai đầu mảng, tần suất bằng n. Actual khớp; đây là trường hợp vùng bằng x chiếm cả mảng.



Hình 4.7 — Biên 3 (BIEN03): toàn mảng bằng x.

4.6. Kiểm tra input không hợp lệ — LOI01

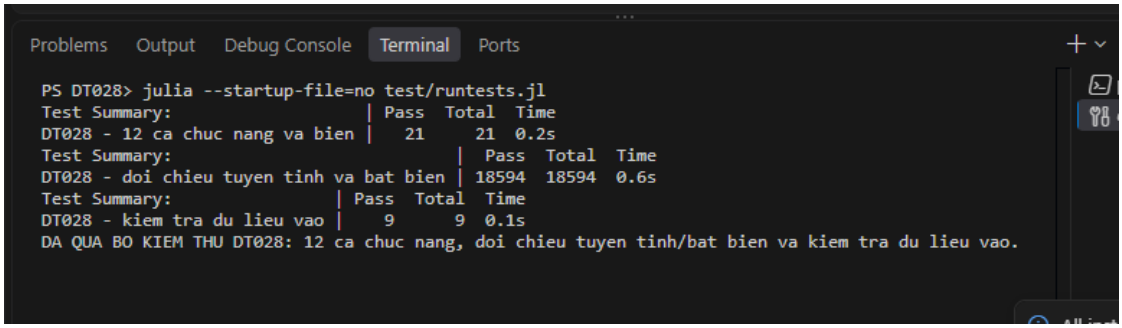
Điều kiện phiếu bị vi phạm: mảng [3,1,2] không tăng; n = 10001 > 10000; x = 2.5 không nguyên. Expected: kiem_tra_dau_vao hoặc kiểm tra x ném ArgumentError kèm lý do, không vào lõi tìm kiếm. Actual: cả ba trường hợp đều bị từ chối đúng.



Hình 4.8 — Input không hợp lệ (LOI01).

4.7. Kết quả toàn bộ kiểm thử

Chạy lại toàn bộ bộ kiểm thử: 3 nhóm, 18.624 phép kiểm đều đạt (21 + 18.594 + 9). Nhóm chức năng kiểm 12 ca chức năng và 1 ca lỗi (LOI01); nhóm đối chiếu kiểm 18.594 bất biến trên mảng ngẫu nhiên; nhóm dữ liệu vào kiểm 9 trường hợp lỗi/hợp lệ. Không còn ca FAIL.



Hình 4.9 — Tổng kết kiểm thử: 18.624/18.624 phép kiểm đạt.

4.8. Khảo sát bốn quy mô hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Khảo sát bốn quy mô n = 5, 50, 500, 5000 trên mảng tăng có nhiều khóa trùng; x lấy tại vị trí giữa mảng để chắc chắn có mặt. Phép đếm là số lần xét mid của từng biên và tổng hai lần; so sánh với số phép so sánh của quét tuyến tính. Thời gian đo bằng time_ns, lấy tốt nhất trong 5 lần chạy.

4.8.2. Kết quả và giới hạn nhận xét

Số lần xét mid tăng theo $\log_2(n)$: 5 → 12 → 18 → 25 khi n tăng 1000 lần. Quét tuyến tính tăng gần tuyến tính (6 → 50 → 480 → 4.781). Kết quả khớp trần $2 \cdot \lceil \log_2(n+1) \rceil$ và xác nhận O(log n) cho cả hai lần tìm.

Bảng 4.4 Khảo sát Q1–Q4

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n = 5, mảng tăng có trùng	quy mô n	xét mid: 3 + 2 = 5; tuyến tính 6	O(1) phụ; khớp trần 6
Q2	n = 50, mảng tăng có trùng	quy mô n	xét mid: 6 + 6 = 12; tuyến tính 50	O(1) phụ; khớp trần 12
Q3	n = 500, mảng tăng có trùng	quy mô n	xét mid: 9 + 9 = 18; tuyến tính 480	O(1) phụ; khớp trần 18

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q4	n = 5000, mảng tăng có trùng	quy mô n	xét mid: 13 + 12 = 25; tuyến tính 4.781	O(1) phụ; dưới trần 26

Dữ liệu tạo bằng Random.seed!(28062026) để tái hiện được. Số liệu thời gian chỉ minh hoạ, không dùng để chứng minh bậc phức tạp; phép đếm là bằng chứng chính.

Với n = 5000, tỉ lệ 4.781/25 gần 191 lần cho thấy lợi ích của tìm kiếm nhị phân khi mảng lớn. Với n = 5, quét tuyến tính có thể nhanh hơn về hằng số, nên chỉ so bậc tăng trưởng.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Repo GitHub đã được đẩy lên tại https://github.com/Mildsivan/ctdl-baocao. Commit C (mã/test/input) có SHA 4277eb8768fbb63ed709df060ecd56e0073346c1; URL và permalink đã được kiểm tra trên trình duyệt. Bảng 4.5 ghi URL, SHA C và permalink; SHA S ghi trong ThongTinNop.txt. Ảnh A10 (clone sạch) và A11 (repo/commit C) trình bày tại Phụ lục E.

Mã nguồn: https://github.com/Mildsivan/ctdl-baocao

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị kiểm lại
URL repo	https://github.com/Mildsivan/ctdl-baocao
Full SHA chạy ảnh C	4277eb8768fbb63ed709df060ecd56e0073346c1
SHA S ghi TXT nộp rời	Ghi trong ThongTinNop.txt (nộp rời, sau commit S)
Permalink hàm chính	https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/src/DeTai.jl
Mã/kiểm thử giữa C và S	Không đổi; đã chạy lại từ clone C, 12/12 ca đạt.
Lệnh chạy lại	julia --startup-file=no demo.jl ; julia --startup-file=no test/runtests.jl

4.9.2. Clone sạch và tái hiện kết quả

Commit C đã được đẩy lên GitHub; ảnh A10/A11 đã bổ sung tại Phụ lục E. Bản nộp cuối được tạo thành commit S; SHA S ghi trong ThongTinNop.txt. Mã/test/input không đổi giữa C và S.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu mục tiêu Chương 1: vi_tri_dau, vi_tri_cuoi, tan_suat, lan_vet_tim_bien, kiem_tra_dau_vao, quet_tuyen_tinh và dem_trong_doan đã cài đặt và chạy đúng; 13 ca có mã PASS (Bảng 4.2, 4.3); khảo sát bốn quy mô khớp O(log n) (Bảng 4.4). Phần Git/GitHub đang được hoàn tất theo mục 4.9.

5.2. Giới hạn và bài học

Giới hạn: thuật toán yêu cầu mảng đã sắp; nếu mảng chưa sắp phải sắp trước với chi phí O(n log n), lớn hơn chi phí tìm. Một lỗi cần tránh là dùng vị trí chèn của x làm first khi x vắng, vì sẽ khiến count sai; ca VD02 được thiết kế để kiểm đúng trường hợp này. Bài học: luôn kiểm tra điều kiện 'có mặt' trước khi ghép hai biên.

5.3. Hướng phát triển phù hợp

Hướng phát triển: (1) phần tùy chọn của phiếu là đếm phần tử trong đoạn [L, U]; đã cài trong dem_trong_doan và không bắt buộc. (2) tìm biên trên mảng giảm hoặc mảng xoay vòng với biến thể nhị phân; (3) dùng tìm kiếm nội suy khi dữ liệu phân bố đều. Các ý (2), (3) là đề xuất, chưa cài.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Hai tìm kiếm biên và tần suất	Hoàn thành	Chương 3, Hình 4.1–4.7	Yêu cầu mảng đã sắp
Đối chứng, đếm phép toán, 4 quy mô	Hoàn thành	Bảng 4.4, log trace	—
Bộ test ≥12 ca, ảnh, mô phỏng HTML	Hoàn thành	Bảng 4.2–4.3; A01–A09	A10–A11 do SV bổ sung

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; Chương 2, Mục 2.6.2; trang in 50–52; trang PDF 62–64.

[2] Mã mẫu BFS của giảng viên (Ma_BFS_mau.zip) — chỉ tham khảo cách tổ chức module, demo, test và chụp ảnh; thuật toán và kiểm thử là tự cài cho đề DT028.

Mỗi nguồn phải được dùng và dẫn trong nội dung; nguồn mã tham khảo ghi rõ phần tiếp nhận và phần tự làm. Không giữ nguồn không sử dụng.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Cây tệp kho mã:

src/DeTai.jl — các hàm tìm biên, tần suất, validator, đối chứng

demo.jl — 8 ca demo + so sánh + nhập bàn phím

test/runtests.jl — 12 ca + 18.594 bất biến + kiểm tra input

scripts/test_catalog.jl, scripts/trace_and_scales.jl

docs/MoPhong_TimBien_DT028.html — trang mô phỏng tương tác

Run_all.ps1 — chạy toàn bộ một lệnh

docs/anh_ket_qua, docs/logs — ảnh và log bằng chứng

Yêu cầu: Julia 1.13.1. Lệnh chạy: julia --startup-file=no demo.jl và test/runtests.jl.

PHỤ LỤC B. Danh mục bằng chứng

Danh mục A01–A09 gồm mã ca, tệp ảnh và log tương ứng; A10–A11 trình bày tại Phụ lục E (Hình 4.10, 4.11). Ảnh trong thân báo cáo phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	01_vi_du_1.png
A02	VD02	02_vi_du_2.png
A03	TK01	03_tu_thiet_ke_1.png
A04	TK02	04_tu_thiet_ke_2.png
A05	BIEN01	05_bien_1.png
A06	BIEN02	06_bien_2.png
A07	BIEN03	07_bien_3.png
A08	LOI01	08_input_khong_hop_le.png
A09	TEST_ALL	09_tong_ket_kiem_thu.png
A10	CLONE	Chưa có — sinh viên bổ sung sau khi đẩy GitHub
A11	GITHUB	Chưa có — sinh viên bổ sung sau khi đẩy GitHub

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

□ MSSV và đề tài đúng bảng phân công (N24DCDT034 / DT028).

□ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.

□ ≥12 ca độc lập; đủ ví dụ và ca phiếu; ảnh A01–A09 là ảnh chạy thật trong VS Code.

□ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và bốn quy mô khảo sát.

Đã có A10/A11, URL, SHA C và permalink; SHA S ghi trong ThongTinNop.txt.

□ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố bài cá nhân: N24DCDT034_DT028_PhanDucHuy.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT034_DT028_PhanDucHuy_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	N24DCDT034_DT028_PhanDucHuy_BaoCao.pdf	Xuất từ chính bản Word nộp.
Mã nguồn	N24DCDT034_DT028_PhanDucHuy_MaNguon.zip	Source, demo, test, README, ảnh/log, mô phỏng HTML.
Thông tin	N24DCDT034_DT028_PhanDucHuy_ThongTinNop.txt	URL GitHub, SHA C/S, permalink, lệnh.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật: https://github.com/Mildsivan/ctdl-baocao

SHA C đầy đủ: 4277eb8768fbb63ed709df060ecd56e0073346c1 SHA S: ghi trong ThongTinNop.txt (nộp rời)

Permalink hàm chính tại C: https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/src/DeTai.jl ; test: https://github.com/Mildsivan/ctdl-baocao/blob/4277eb8768fbb63ed709df060ecd56e0073346c1/test/runtests.jl

Môi trường và phiên bản: Julia 1.13.1, VS Code 1.142.0, Windows 11

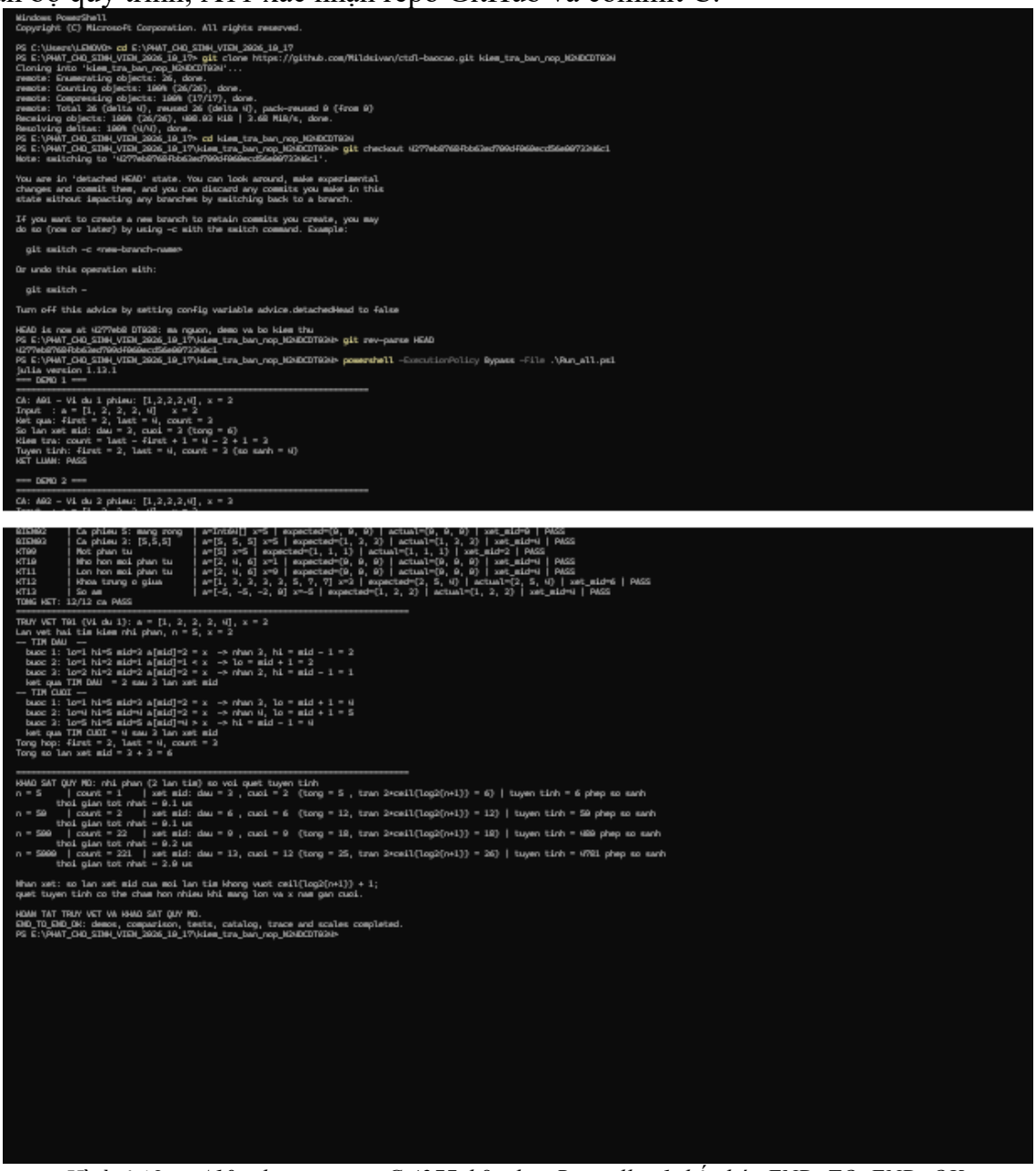
Lệnh chạy hai ví dụ và bộ test: julia --startup-file=no demo.jl --ca=1/--ca=2; julia --startup-file=no test/runtests.jl

Kết quả clone S: ghi trong ThongTinNop.txt (sau khi chốt S) Số ca và phép kiểm: 13 ca / 18.624 phép, 13 PASS

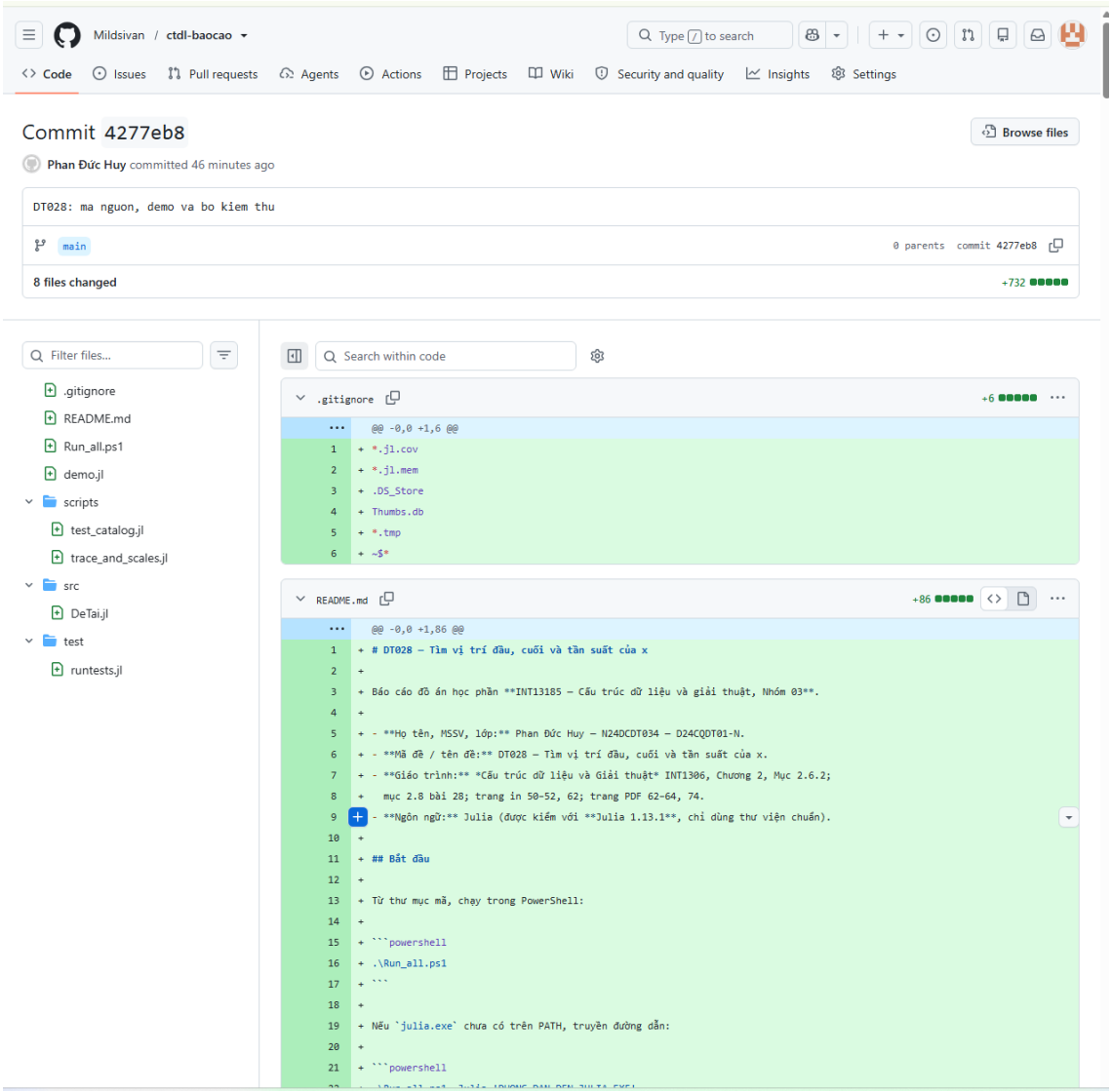
Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.

PHỤ LỤC E — ẢNH MINH CHỨNG A10 VÀ A11

Hai ảnh dưới đây được chụp từ lần chạy thật: A10 kiểm tra clone commit C và chạy lại toàn bộ quy trình; A11 xác nhận repo GitHub và commit C.



Hình 4.10 — A10: clone commit C 4277eb8, chạy Run_all.ps1, kết thúc END_TO_END_OK.



Hình 4.11 — A11: repo GitHub Mildsivan/ctdl-baocao tại commit C 4277eb8.